

om Jeff Anderson's talk.

## Improving the Customer Experience Through Ticket Deflection

"Ticket deflection" can sound like a way to weasel out of work, but in reality it's about improving the customer experience.

Customers don't want to write into support. They'd much rather not have the problem in the first place.

Failing that, they'd like to solve the problem themselves. If they can't do that, then they want a technically skilled individual to help them solve their problem.

There are 4 primary tools of ticket deflection:

- Excellent Product
- Statement of Support
- Documentation
- Technical Excellence

In essence, at the end of every ticket, there should be a link to documentation, an issue, a merge request, or our statement of support.

### Excellent Product

Having an excellent product is the first line of deflection - a product that is free of defects and works as expected will reduce the number of support cases organically.

Support plays a vital role in surfacing issues that our users encounter while using GitLab by:

- reporting bugs
- tagging issues
- participating in issues
- surfacing feedback
- fixing issues by submitting an MR

### Statement of Support

The Statement of Support describes the areas covered by Support and the areas we can't promise to cover. This is a tool to set expectations for customers as well as to help the support team to make sure we're supporting what we're experts in. You can read more about the philosophy behind it in the blog post where we introduced the Statement of Support.

As a member of the Support Team at GitLab you should be:

- familiar with the content of the Statement of Support
- comfortable explaining to customers when something is out of scope
- cognizant of when you're intentionally going out of scope and conscious of articulating to the customer that you're doing so "as a courtesy"

Is it in scope?

Greg's razor is a simple question that will help you determine what is in the scope of support.

> Is it in the docs?

If yes, we support it.

If it's not in our docs, the first step before any customer uses it in production should be to get it in the docs.

## Documentation

By taking a docs-first approach to answering, we can ensure that the documentation remains a highly useful single source of truth. By building up a corpus of documentation informed by real-world problems, we help GitLab customers find the answers or solutions they need before they come into the queues.

Always respond with a link to the docs. If docs content is missing, create it and link the customer to the MR. If you are working on a ticket that is about to breach, you can clear the breach with a response, and immediately follow up with the MR. Remember: go slow, to go fast.

## Technical Excellence

The best way to improve the customer experience is by being knowledgeable about our product. You should coordinate with your manager to develop an intentional learning plan that builds on your strengths or broadens your knowledge. You should also freely ask questions, pair with others and demonstrate a posture of vulnerability that invites others to follow.

Whatever you learn, make sure that you're constantly bubbling it up and broadcasting it:

- When learning: (re)write the docs.
- When troubleshooting: use the docs.
- If something is missing: update the docs.

## Highlighting docs and handbook links on our support portal

At times, we might want to highlight a GitLab document or a handbook article on our support portal page. We have the facilities to create a redirect article in Zendesk and associate certain keywords with this link (while pointing to the relevant doc or handbook link). During the creation of a support ticket, this article will pop up if the keywords mentioned above are used in the ticket subject, allowing the customer to see the answer to their question before submitting the support ticket.

Currently we are curating the list of articles and redirects, so you need to reach out to Support-Ops or the managers to iterate on the (list of) articles.

Openly share your mistakes and learn from them

We are all human and while we all strive to ensure our interactions with customers are 100% correct, the truth is that we are bound to make mistakes from time to time. This can create a stressful or anxious situation if you provided a customer with the wrong advice, or you were unaware of a particular aspect of a ticket that was pointed out to you later on for example.

Regardless of the situation, if you make a mistake, take ownership of it and learn from it. Remember our transparency value.

While the immediate situation might be less than desirable, once you resolve the situation it can be very empowering. If you don't know how to resolve the situation, don't be afraid to ask for help, everyone is here to assist. When you follow up with the customer, be sincere and do explain to them that you made a mistake and provide them with the correct information.

Once the situation is resolved, take time to reflect on your actions and what you could do next time to mitigate that situation from happening again.

If you feel that is something the wider support team can learn from, please share your experience in your regional support team meeting and/or in the Support Week in Review (SWIR).

If appropriate, ensure that you make a merge request to our Support documentation as well. By sharing your experiences, others can contribute to other methods on how they might have dealt with your situation and it also makes them aware, so they are less likely to repeat that same mistake themselves.

---

## SECTION: support/workflows/how-to-set-up-shift-coverage-for-an-event.md

## Overview

Use this workflow when you want to get a coverage schedule, including PD, going for an event like Contribute

---

## Workflow

### Building a Shift Schedule

Take a look at this example

1. Determine the destination timezone and how it maps to the regions of support
1. Determine who will be attending the event and who will remain in their region.
1. Based on the above, build a spreadsheet with an appropriate number of shifts to get coverage for regions
  1. Shift length could be variable, but 2 hours seems to have worked well.
  1. Number of people per shift will have to be determined by the coverage gaps

## Building a PD Schedule

- People on shift will handle emergencies
- It may be preferable to create a PD schedule to help facilitate emergency coverage.

Build a PD schedule:

1. Create a new rotation with shift lengths equal to the coverage sheet
1. Add managers in and round-robin through them
  1. Pay attention to the hours of the rotation - it may not be necessary to have 24 hour coverage if some people are in place

Build an Escalation Policy:

1. Escalate first to the PD schedule you just created
1. After some minutes escalate to the team members present at the event
1. Finally, escalate to the 'normal' PD rotation

Updated the Customer Emergencies Service:

1. Update the "Customer Emergencies" service to use the temporary escalation policy
1. Test it out by initiating an emergency

---

## SECTION: support/workflows/how-to-wir-podcast.md

## Overview

Use this workflow as a general guide when you want to record and publish a Support Week-in-Review Podcast.

Much of this is done in the Support Week in Review Project. For more information about the CI/CD jobs used in this guide and what they do, refer to the project readme.

---

## Workflow

1. Check the Support Team Google calendar for the recording session (near the end of the week).
1. Determine who will be responsible for each of the following roles. One person can be responsible for them all.
  - Editor:
    - Before the Recording:
      - Prepare the SSAT input
        - SSAT is gathered into the SSAT issue in the SWIR project.
        - Some SSAT input will have been provided by managers during the week, but most will need to be ingested using the populatessat job from the pipeline.
        - Run populatessat job - this will gather any open positive SSAT into the SSAT issue

- Review the content (automated or other) and make corrections and remove anything that is not actually positive. If there is a lot of content, consider reducing the number down by removing some that are short and not personalised - use your judgement here. Remove the line about "automated content".

- Prepare the digest issue
  - run the createdigestissue job.
  - Edit the digest issue and add a phrase of the week
  - Work with the Metrics Analyst (below) to ensure the metrics section is populated
- After the recording
  - Publish the podcast
  - Prepare SWIR for the next week
- Metrics analyst:
  - take screenshots of the key metrics from the MM: Support KPIs Zendesk dashboard and insert them into the SWIR digest issue. The following 4 items need to be included:
    - From the SWIR tab: the top row of 4 graphs, from SSAT through to volume
    - From the SWIR tab: scroll to the bottom of Total FRT SLA achievement - Last 4 Weeks and capture the current week
    - From the SWIR L&R tab: the top row of 4 graphs, from SSAT through to volume
    - From the SWIR L&R tab: scroll to the bottom of L&R FRT SLA achievement - Last 4 Weeks and capture the current week
  - in text, enter key metrics into the appropriate sections of the SWIR digest issue
  - you can use the following for formatting

markdown

SSAT:

FRT:

High:

Normal:

Low:

NRT:

- Copy US Gov stats from this doc from earlier in the week
  - gather pairing data by observing the number of issues against the current pairing milestone and compare to the previous week's digest issue data for the week on week (WoW) metric.
    - read and record this section during the recording call.
  - Narrator(s):
    - analyze the content of the section you'll be narrating: click on each link and understand what is being expressed by the point
    - read and record the content of the point
    - by convention, we verbalize issue and MR links by reading out their project name and number, such as "Support Team Meta 1234", or "Handbook MR 4321". You can see these while narrating by hovering over the link. This is particularly important when the item has a "here-link" such as "see this issue".
1. Join the Zoom room
  1. Determine speaking order for Narrators. A useful set of conventions is:
    - Read in alphabetical order by your first initials.
    - If you have an item and it comes up, you will read it which will reset the order.

1. When everyone is ready, begin recording. It's easiest for the Editor to "Record locally", as they'll have the audio on their computer for upload.
1. When finished recording, the Editor will publish and prepare the project for the next week.

### Publishing the Podcast

Once you have the compiled audio:

1. Optional: add the theme music to the recording if you have it
1. Upload it to the Support Week in Review - Audio Edition folder
1. Change the Sharing Settings to "Anyone within GitLab can View"
1. Copy/paste the URL into the Digest Issue below the TOC (use the format: :speaker: Audio edition)
1. Share the final mix into Slack (there is a slackbot reminder in supportteam-chat at 23:00 UTC on a Thursday - you can share the audio link as a threaded reply to that conversation)

### Prepare SWIR for the next week

1. Run the closeweekandcreatenewmilestone job
1. You're done!

### Notes on SSAT content

The purpose of including SSAT content in the Support Week in Review is two-fold:

1. To highlight behaviors and ways of interaction that customers value in their own voices.
1. To highlight the excellent work on the part of individuals and groups of individuals that take place in ticket interactions.

We do not include every SSAT review every week, both for brevity and because not every SSAT review fulfills the purposes above.

With the above purposes in mind, SSAT comments in the Support Week in Review should:

- be truly positive! Avoid SSAT ratings that disparage other engineers, GitLab teams or are back-handed compliments that include a mix of praise and negative feedback.
- have significant, thoughtful or interesting comments.
- be from a mix of Engineers. Sometimes a particular engineer may have multiple positive reviews in a week: try to get a cross-section of the team included.
- highlight the accomplishments of new team members when possible: a first positive SSAT in the SWIR should be called out and celebrated!

---

## SECTION: support/workflows/improving-documentation.md

### Improving Documentation

As outlined in the Documentation section of the How to Respond to Tickets workflow, comprehensive documentation is a powerful tool for ticket deflection.

> By taking a docs-first approach to answering, we can ensure that the documentation remains a highly useful single source of truth. By building up a corpus of documentation informed by real-world problems, we help GitLab customers find the answers or solutions they need before they come into the queues.

With that in mind, whenever possible, include a link to the applicable documentation as part of your response to a ticket. If the documentation does not exist yet, then write a documentation issue or draft merge request, and send the link in the response.

The information collected below should be helpful for when you contribute to the docs.

## General Documentation Guidelines

### Making Your Changes

Make your edits and commits on a new branch named according to the Documentation guidelines, which state that branch names must start with docs- (or one of the other options listed in the branch name scheme).

- Commit the changes. Make sure the commit message follows our commit message guidelines.

If you don't follow the guidelines, the Danger Bot job may fail when it checks if the commits are aligned with our guidelines. If it fails, read the job trace and fix the issue with a new commit, or an interactive rebase. (Ask for help if you're not sure how to do a rebase!)

> Note: If you are unfamiliar with the basics of working in a Git or GitLab environment, please complete the Git and GitLab Basics training module before attempting to make documentation changes.

### Style Guide and Linting

- Review the documentation style guide.

- It's not required to memorize the style guide, but remember to check it when writing your documentation to ensure it complies. This will help to ensure your Merge Request is approved quickly.

- Remember to also review the topic types page, paying close attention to the troubleshooting section.

- While not required, it is highly recommended to install the docs linters in your local editor to prevent pipelines from failing, or use the GDK. Plugins can be also used from the command line.

- Set up markdownlint.

- Set up Vale.

- Set up a vertical ruler to help you split long lines.

### Documentation Merge Requests

- Follow the docs MR guidelines as you create the MR

- In the MR Description, be sure to include a link to any relevant ticket or issue

- Add appropriate labels:

- documentation (the template should be automatically adding this label, but check to make sure).

- group scoped label (if in doubt, it should be listed at the top of the docs page)
- customer (if related to a ticket)
- Note: the bot will add the stage, section, Support Team Contributions, and docs-only labels if you don't
- Follow the Code Review Guidelines and set the relevant Technical Writer (TW) as a Reviewer.
  - The documentation template has instructions on how to find this, but basically refer to the metadata information at the top of the source version of the docs page.
  - Check the Technical Writing Assignments to find the appropriate tech writer for the page you edited.
  - If you're using the Support dotfiles, you can use the find-technical-writer command to quickly see who is the tech writer for a specific group.
- Make sure that the following options are checked before submitting the MR:
  - Delete source branch when merge request is accepted.
  - Squash commits when merge request is accepted. The "Apply suggestion" feature is used frequently in docs MRs, so this checkbox is essential to keeping a clean history in the default branch after merging.

### Working with the Tech Writers on the Merge Request

- Remember to provide timely answers to any questions the tech writing team may ask you about your contribution.
- Read through the documentation process.
  - Note: Support is expected to verify the technical accuracy of a docs MR before assigning to a TW. When in doubt, get a technical review from an engineer first. Find an appropriate engineer using the metadata information from the edited page.
- Read the post merge review guidelines.
  - In Support, you might use the post-merge review option if you have an urgent docs MR, such as when we need to publish a docs fix as soon as possible.
  - If it's in the "Troubleshooting section" of any page, follow the guidelines except assign to a support manager who is online who will do a quick review and merge.
  - If it's not, then depending on how quickly you need it, share it in the docs Slack channel asking anyone there to review and merge ASAP. This does not require a post-merge review, as it involves a TW review.
- Read about docs deploy. The key thing is to note how often docs are deployed.
  - Optionally, read the rest of the site architecture information to learn how the docs site is built.
- Also remember to not take feedback personally - we all want the documentation to be accurate and easy to follow, so all feedback from the tech writers will be with this goal in mind!

### Zendesk "Document this" workflow

For those situations in which making documentation is a more time-consuming exercise, we have set up a process to automatically create an issue in the GitLab issue tracker to track documentation tasks that needs to be completed:

- Apply the macro General::Document This to the ticket. This macro will apply the createdocissue tag and prefill the template for you to fill out.
- Fill out the template changing the title and the description. Example of the filled out template:



text

## CREATE DOCUMENTATION ISSUE

Title: Document how to use Y with an X

Description

There is a new cool feature Y that we released last month. It can be used with X to improve Z. We need to document how Y can be used with X.

Test h2 header

We should remember to document:

1. requirements
1. how to enable
1. how to configure
1. how to troubleshoot

- Submit as internal note. When issue is created docissuecreated tag will be added to the ticket.

If you need to create multiple issues using this workflow, just apply the macro once again. The automation creates issues with `labelname\[ ]=documentation&labelname\[ ]=customer` and assigns them to the agent who submitted the internal note.

Note: Make sure to use the template so that your internal note is correctly parsed by the trigger.

This workflow is implemented using a zendesk trigger that parses the internal note and sends it to Zendesk's HTTP target that creates a documentation issue.

---

## SECTION: support/workflows/increase\_compute\_minutes\_trial.md

## Overview

NOTE: This workflow is DEPRECATED.

Use this temporary workflow when a user reports that they need more than the 400 minutes currently being allocated for Trial subscriptions. In most cases, 10000 minutes should be the target. This is only temporary until issue 322043 has been applied.

---

## Workflow

1. Confirm the namespace that is under a Trial subscription account.  
Note: compute minutes should reflect 400 minutes.
1. Confirm the user requesting additional minutes is a member of that namespace.
1. Update the minutes to reflect 10000, instead of 400.

1. You can update the minutes as a .com admin, or
1. Use /chatops to set additional minutes quota for a namespace.

For more details on adding more minutes to an account, see [Adding additional compute minutes](#).

---

## SECTION: support/workflows/information-request.md

There may be times when a request comes in that seems to need input from our Legal team. This workflow explains how to handle such a request when it does not fit into one of the prescribed workflows.

## Workflows

### Incoming Subpoena or court order

From time to time we may receive subpoenas or other legal requests for information about GitLab users, their data or activity.

This workflow clarifies how to handle these requests and subsequent workflow related to delivering information if our counsel and CISO approve.

Note: If a ticket comes in that you believe is Legal related, but is not a subpoena or court order, please check the list of other legal workflows.

If a subpoena or court order that is not covered by another workflow (e.g. DMCA, GDPR, information request) comes in:

1. Forward the email to <legal@gitlab.com>
1. Reach out on the Slack legal channel to inform the Legal team.
1. Respond to the ticket with the Legal::Subpoena or court order macro.

### GitLab.com specific support policies

These workflows should be used when a user question is addressed in a specific Support policy.

### Namespace and Trademark claims

Respond to the ticket with the Legal::Namespace and Trademark macro.

### Ownership Disputes

Note: This workflow is specifically related to inquiries about disputes on ownership. You can also check other ownership-related workflows to ensure you are using the correct one.

Respond to the ticket with the Legal::Ownership disputes macro.

### Log Requests

Please see Log requests workflow

### Other Legal-related Questions

A request may come in that you believe needs legal input but does not fit into one of the prescribed workflows. Follow this workflow in such a situation:

1. Follow the instructions for contacting Legal based on the specifics of the request.
1. Respond to the ticket with the Legal::General macro
1. If ticket has not been resolved within 4 days (when ticket will auto-resurrect), check in with Legal for an answer.
1. If Legal requests more time, respond to the ticket with the Legal::Handover macro.
1. Note the above response in the related Legal and Compliance issue.
1. Close ticket.

Unsure? Other?

Support should be handling responses to any inquiries for clarifications. In most cases, an existing customer will be referred to their account manager.

If you're unsure on how to respond, post in the supportleadership Slack channel for guidance.

---

## SECTION: support/workflows/internal\_requests.md

NOTE: If you are a GitLab team member who wants to file an internal request, please see the Support Internal Requests handbook page.

NOTE: For internal requests relating to licenses and subscriptions, please refer to the relevant license and subscription workflow, or CustomersDot console workflow.

### Overview

This document details the various templates and workflows that should be followed in order to properly handle GitLab.com related requests that Support receives in the internal-requests issue tracker.

Internal requests can and should be addressed by any GitLab Support team member who is able to resolve the request.

Those who are at least 50% SaaS focused and have a GitLab.com administrator account should subscribe to and handle any issue that is an Admin Escalation.

Internal requests are typically created by other team members who are not within the Support organization, but you can always create an internal request to track work being done, especially in cases where the request originates internally (and there is no ZenDesk ticket to track).

At the very least, you should subscribe to the following labels:

- Admin Escalation
- Dotcom Escalation Weekly Report

You may want to consider subscribing to Platform::SaaS but be aware this will be noisier.

By subscribing to the labels, you'll receive notifications on when a request is created. You should try to work them into your regular workflow, ensuring that you are assigning it to yourself like a customer ticket if you decide to take it.

If you are interested in servicing internal requests that require console access, consider speaking with your manager about completing the GitLab.com Console module.

### General Workflow

```
mermaid
graph TD;
  A[Issue Created]-->B[Assign Yourself];
  B-->C[Issue Actionable?];
  C-->J[No];
  J-->K[Apply Status::Blocked and Inform Submitter];
  C-->D[Yes];
  D-->E[Take Action];
  E-->F[Issue Resolved?];
  F-->G[Yes];
  G-->H[Close, or Apply Status::Awaiting Confirmation and inform submitter];
  F-->I[No];
  I-->L[Requires waiting?];
  L-->M[Yes];
  M-->N[Apply Status::On-Hold and set due date];
  L-->O[No];
  O-->P[Escalate as necessary];
```

### Activate Trial Runners

For sales assisted trials, only we can override the credit card validation requirement for a namespace. See Internal Requests > SaaS Trial Related > Change an existing SaaS trial plan for instructions on how to perform this task.

### GitLab.com Trial Change Request

Sales team members will typically open this on behalf of their prospects in order to extend an active trial. You can follow the L&R Workflow for Extending Trials.

### Ask for Support as a Solution Architect

Solution Architects should use the Support Super form only when support is needed during pre-sales (for example: a POV exercise, or a potential new deal with an existing customer). This

form is an internal request, which means the work takes place between the SA and the engineer. Customers can not collaborate on these tickets. In any other cases, SAs should advise the customer to open a support ticket themselves, for clear and efficient communication.

### Inactive Namespace Request

GitLab Team Members can no longer submit inactive namespace / namesquatting requests for their own use. See: support-team-meta5170

Customers can submit a support ticket. See Name Squatting Policy.

### Contact Request

GitLab team members should use the Contact Request template to request Support to contact a user on their behalf.

Usually, these are assigned to CMOC, but can also be completed by anyone with admin-level access to GitLab.com. Admin Escalation label is applied and everyone with admin access is supposed to be subscribed and working on such issues.

See the Sending notices workflow for complete instructions on how to contact users.

### Repo Size Limit Increases

At times, users require an increase to the size limit of their repository as a workaround for a bug issue or due to having inadvertently reached the limit. There are two different limits in place, 10GiB for free and 500GiB for paid namespaces. In both cases, we should work with the users to reduce their usage as soon as possible. Free users should be encouraged to purchase additional storage packs, while paid customers should be directed to speak with their account managers for next steps.

In both cases, the limit can be temporarily increased:

1. Open an issue in the internal-requests issue tracker using the Repo Size Limit issue template.

- If you do not have GitLab.com admin access, add the Admin escalation label. If needed, post in the supportgitlab-com Slack channel for attention on it.

1. If a request is for a free user namespace or to work around a bug issue:

- Set the expectation for free users that this is very time limited, just enough to allow them to reduce usage or purchase additional storage packs

- Set a due date for reverting: 1-2 days for free users

- up to 1 week from the current date for bug encounters

- If a longer exception time is required, add the Manager Approval::Required, and post in the supportleadership channel to request approval.

- Use current size + small buffer (2-5 GB) for the exception size.

- Make sure to comment on (or create) a bug issue to help future users and prevent further tickets.

1. If a request is for a paid namespace:

- For system stability reasons, we should encourage them to reduce usage as soon as possible

- At this time, it is not possible to purchase additional storage beyond the fixed project limit
  - Inform their account management team if they aren't already involved, and reach out to support/licensing-subscription if you need any assistance in locating or communicating with the account managers
  - Set a due date for reverting, up to 1 week from current date, but loop in support leadership if longer will be necessary
1. Apply the Status::On Hold label and set the due date to when it should be reverted.
  1. Using your GitLab.com admin account navigate to the project in question while appending /edit to the URL. For example, if the project in question is located at <https://gitlab.com/group/subgroup/project/> you would navigate to <https://gitlab.com/group/subgroup/project/edit>.
  1. Enter a new value in the Repository size limit (MB) field.
  1. Click Save changes.
  1. On the specified due date, revert the size limit back to the default by removing the value.

### Pipeline Quota Reset

See internal wiki page.

### GitLab.com Console Escalation

Templates available for requesting an engineer with GitLab.com console access to take action:

1. Console Export Request for creating a project export with the console.
1. Console Escalation for running commands in the rails console.

Before submitting a console escalation request, ensure that you perform Account Ownership Verification workflow for the following types of customer request:

1. To extract and release information.
1. To make changes to account(s).

This ensures that the request is coming from an authorized contact. Console escalation requests for internal investigation do not require account verification. See Support Team Meta 5276 on the discussion.

Common issues include the following when the UI and API methods are not working:

- Deletion of projects, groups, cluster integrations, container registry images, etc.
- User account confirmation retrigger
- Project export

Rarely, console escalations will also be used to workaround lack of a feature where Support can complete actions that would be unavailable or onerously time-consuming to customers. When actions cannot be done via the UI or API, engineers should ensure that a feature request exists to have it in the product. Consider tagging the appropriate Product Manager for prioritization.

Console escalation requests can also serve a purpose when further information (unavailable through the UI or API) is needed to understand the root cause of a problem. For example,

searching for unverified secondary email until gitlab367823 is resolved. This may be because we are not sufficiently logging in Kibana/Sentry, we're unable to replicate an issue, or the creation of an issue may not be the appropriate action needed to resolve a customer problem. Collaborate with console enabled engineers and product teams to solve these types of problems.

Any request requiring assistance from infrastructure teams, please see the SaaS Platforms Request for Help issue tracker and SaaS Platforms Getting Assistance page.

### Response time and escalating the request

Support engineers with console training and access are subscribed to the "Console Escalation::GitLab.com" label, and one of them should respond within 1 business day. If you need to reach out to the group, you can do so by mentioning the console group. Please note that there is a read-write group for requests that require write access. If it's urgent, please ping the specific team members who are working based on timezone in the supportgitlab-com Slack channel. If none are available, please engage infra SRE on-call and notify the Support Manager on-call for awareness.

### Fulfilling console requests

Engineers with console access should search for similar previous requests, look for the relevant function in the code, or work with another engineer to resolve each request. You can filter requests by "Read::Only" or "Read::Write" labels. Common or custom functions can be found in the support runbooks.

For update, create, and delete actions, it's essential to carefully review the request and think about the impact of these actions. Remember, update and delete actions can be risky. When writing custom commands or scripts, it's crucial to calibrate them based on the potential risks and the situation's specifics. You should always get another set of eyes on your code to confirm what you want to achieve. Customer data loss is not an acceptable tradeoff for expediency.

If you're not completely sure about the specifics of the script or commands, test them out in your test instance first. And if needed, get feedback from developers who know that area of codebase before using them in the production console. This approach helps reduce risks when doing console tasks in production. For any scripts that are originating from the support team and are not already in a runbook or approved by development teams, please reach out to a manager or staff support engineer for review and approval.

### CI Catalog Badge requests

CI Catalog Badge requests are to be actioned by a Support Stable Counterpart for Pipeline Authoring. These requests are used to give certain organizations a "Partner badge" in the CI catalog on GitLab.com. They require a GitLab.com administrator account to execute the `verifiedNamespaceCreate GraphQL` mutation.

1. Open GraphQL with your GitLab.com administrator account
1. In the following query, replace root-level-group with the namespace and verification level (GITLABPARTNERMAINTAINED, VERIFIEDCREATORMAINTAINED) provided in the internal request:

```

graphql
mutation {
  verifiedNamespaceCreate(input: { namespacePath: "root-level-group",
    verificationLevel: GITLABPARTNERMAINTAINED
  }) {
    errors
  }
}

```

1. Execute the query via GraphiQL

- In case of an error, reach out in gpipeline-authoring for assistance

1. When closing the internal request as actioned on, ping the requester to let them know the badge was applied

---

## SECTION: support/workflows/investigate\_blocked\_pipeline.md

## Overview

GitLab.com uses an external pipeline validation service to verify pipelines as legitimate activity. This includes pipelines run on specific runners. If a customer is reporting that their pipelines are not running we can review the logs on our end to see what activity was logged in the validation service.

Because of the sensitive nature of the rules we use to detect abuse, in most cases we can provide only general feedback to the customer as to what in their .gitlab-ci.yml is causing their pipelines to be blocked. When in doubt, request @trust-and-safety to review.

What does it look like for a user?

A pipeline may have been blocked by the pipeline validation service if an expected pipeline is not created. A pipeline might be generated via:

- a new push
- a manual action
- a scheduled pipeline
- the API

There may be a different underlying cause though, so it's important to validate by searching the logs.

In some cases a user may see an error:

- > Pipeline cannot be run.
- > External validation failed.

Identifying a block



1. Identify the affected projectid or userid
1. Search Kibana in the pubsub-pvs-inf-gprd index in the appropriate time frame for:
  - json.jsonPayload.projectid: <projectid>
  - json.jsonPayload.userid: <userid>
1. Observe the msg and rejectionhint fields to understand why the pipeline was blocked.

For more, see the pvs-runbook

The rules are documented in the Trust and Safety - Repository Validation Service repository (GitLab internal only)

Trends / High Priority cases

When rules change or are updated there's the potential for a batch of false positives. If there are many reported cases in a short period, report an incident.

It will be helpful to:

- have a list of tickets reporting blocked pipelines
- verify that they were blocked by the pipeline validation service
- identify any commonalities

Responding to Customers

We have the General::Blocked Pipeline macro which will ask users for their .gitlab-ci.yml (or permission to access it) and explains that their pipeline may have been blocked, in addition to tagging the ticket. Make sure to remove any items the customer has already provided in their initial response.

Once you've verified the block you can respond with positive affirmation that the pipeline was blocked as a result of detected abuse.

In responding avoid mentioning any of the specific rules. For example, say we had a rule that filtered out imaginary crypto mining script use-gitlab-to-mine-crypto.sh. You might respond:

> It looks like an item in the script section of your .gitlab-ci.yml tripped our anti-mining measures. In general, you'll want to remove any references to mining tools.

or

> I see you've got an item on line 22 that calls a script with a name similar to scripts that have been flagged as abuse in the past. If you remove that line, I suspect your pipeline will pass validation.

You would not want to say:

> We scan all pipelines for the string use-gitlab-to-mine-crypto.sh. Your .gitlab-ci.yml contains this string, so please rename that script to avoid being flagged.

## Reporting a false positive

Create an issue with Trust and Safety. Include the username of the affected user, a copy of the JSON output from Kibana that shows the block happening, and a link to the Zendesk ticket reporting the request.

---

## SECTION: support/workflows/investigate\_commits.md

### Overview

Users occasionally write in to say that there is a commit from the incorrect or an unknown user or email.

It is part of Support's responsibility to determine whether this is due to a misconfiguration (which is common) or a true security incident.

### Finding the commit user

For more information on using Kibana in general, please see 500 errors workflow.

To find the user who made the commit:

1. Open sidekiq logs.
1. Search for the commit SHA.
1. Optionally add project path as json.meta.project.
1. Look for an entry with an insertId and the SHA under json.args.
1. The json.meta.user indicates the GitLab username that pushed the commit.

If logs are unavailable in Kibana (older than 7 days), try searching the project's activity page for the SHA. If that doesn't work, post in Slack security for assistance.

### Check Authorization

Based on the search results, check to see if the user is authorized to have access to the project.

If yes, then respond to customer clarifying which user made the commit and for the user to double check their gitconfig name and email address.

If not, then open a secops issue for further investigation.

---

## SECTION: support/workflows/ip-blocks.md

Users of GitLab.com can find that their IP address has been blocked due to rate limiting. Currently, rate limit parameters on GitLab.com are best described on the GitLab.com settings docs page. The single source of truth for rate limiting on GitLab.com exists in our

infrastructure rate limiting documentation. This page is intended to help Support Engineers troubleshoot issues related to rate limits and IP blocks.

## Responding

A standard response is available as a macro: Support::SaaS::Gitlab.com::Temp IP Ban

Please also see the log requests workflow for what information we can provide when responding.

## Searching

### Search Condition

Start by adding a positive filter on `json.meta.remoteip` for the IP address provided by the user:

You can then drill down from there with positive and negative filters on fields to get the best results.

### Checking for Rack Attack Blocks

If a user has been blocked by Rack Attack, we should be able to locate requests in Kibana that were blocked because of it.

To do so, enter the IP address into the main search field and set a positive filter on `json.message` for RackAttack.

You should see results similar to the following:

The existence of these results tells us that this user was blocked by Rack Attack and we can add the `json.fullpath` field to see which exact path on GitLab.com each request tried to access.

It's common to see multiple failed authentication requests (401) trigger a Rack Attack block which causes a 403 Forbidden message. We block IP addresses if we receive 300 failed requests from a single IP in a one minute period. It's worth noting that by default, Git operations are first tried unauthenticated so it's expected to see two 401 responses for every Git operation.

Rack Attack can also throttle traffic. If that is the case, this is recognizable by the HTTP 429 response code. The preferred solution to this is to have the user make fewer requests. Please refer to our Bypass Policy for more information.

### Checking the Application Rate Limiter

If you can't find a block in Rack Attack, try checking the application rate limits. It's

possible to search this in Kibana where `json.message.keyword` is `ApplicationRatelimiterRequest`. You'll be able to identify the different limits with `json.env` and still be able to filter by `json.meta.user` and `json.meta.remoteip`.

## Fields

### Primary

The following fields are the best to add to your search query in order to get the most important details on multiple requests at a glance.

- `json.status` - Outputs the HTTP status code that was returned for the request. We're usually looking for 401 (Unauthorized) and/or 403 (Forbidden).
- `json.path` - The path on GitLab.com that was accessed by the request or the API endpoint that was hit.
- `json.method` - Can be either GET, POST, PUT, PATCH, or DELETE. The first three are the most common.
- `json.env` - Can be blacklist, throttle or track. track is used for diagnostics when changing rate limiting settings and does not affect users: from a support perspective, ignore track. blacklist happens in reaction to too many failed authentication attempts, for example with automated Git HTTP traffic. throttle means a user or IP is making too many requests per minute.

### Secondary

These fields can be helpful but aren't essential.

- `json.controller` - Gives you a clue as to what part of GitLab.com was being accessed by a particular request.
- `json.params` - Shows what user made the request, what action was taken, and on what resource it was taken on. This field shows what repository was targeted for requests to the container registry.
- `json.matched` - This is the name of the Rack Attack rule used to limit this request (when `json.env` is throttle). This can help you find out which of the current limits the request surpassed.

## Common Causes

### Container Registry

Numerous failed pushes or pulls to `registry.gitlab.com` can result in an IP block, from the docs:

> GitLab.com responds with HTTP status code 403 for 1 hour, if 30 failed authentication requests were received in a 3-minute period from a single IP address.

You can list all log results for hits on the container registry by searching for the provided IP address and setting a positive filter on `json.path` for `/jwt/auth`.

## Useful Fields

- json.status
- json.path
- json.params
- json.controller

## Failed Push and Pull Examples

### Push

Failed pushes to the registry will always have JwtController for the json.controller field and /jwt/auth for the json.path field. Watch for :push,pull in the json.params field, indicating that the request is for a push.

A failed push will look like the following in Kibana.

### Pull

Similar to a push, failed pulls from the registry will always have JwtController for the json.controller field and /jwt/auth for the json.path field. However, in json.params only :pull will be present.

A failed pull will look like the following in Kibana.

## gitlab-ci-token Pulls

The gitlab-ci-token user is exempted from rate-limiting.

## LFS

Pushes or pulls to repositories containing LFS objects can result in an IP block if the user is unauthorized.

## Useful Fields

- json.action
- json.controller
- json.method
- json.status

## Examples

### Push

Failed LFS pushes will always have uploadauthorize in the json.action field,

Projects::LfsStorageController for the json.controller field, and PUT for json.method.

A failed LFS push will look like the following in Kibana.

### Cloning Fails

An IP can become blocked if a repository is cloned via HTTPS without authentication enough times.

The request will look like this:

### Useful Fields

- json.status: 401
- json.controller: Projects::GitHttpController
- json.action: inforefs
- json.path: /namespace/project.git/info/refs
- json.ua: If the request did not come from a browser this will reveal the operating system of the machine that attempted the clone along with the version of Git installed on it. Examples: git/2.11.1.windows.1, git/2.17.2 (Apple Git-113), git/2.17.1 (Ubuntu 18.04 LTS), etc.

You should also see git-upload-pack in the json.params field.

### Project Export Rate-Limiting

An IP can become rate-limited if a customer attempts to export or download project exports too rapidly. See Project Import/Export

### Useful Fields

- json.method: ApplicationRateLimiterRequest
- json.controller: ProjectsController
- json.action: downloadexport
- json.path: /namespace/project/downloadexport

### Email verification process

In certain cases, when the customer is using a shared user account to run pipelines, a signing sign in from a new IP address will trigger Account email verification. this will block the

account, and all tokens, until the signing is verified. This could cause enough 401 errors to trigger an IP block.

#### Useful Fields

- json.details.custommessage : User access locked - sign in from untrusted IP address
- json.custommessage: User access locked - sign in from untrusted IP address
- json.entitypath - The user name of the account

#### Cloudflare troubleshooting

There may be cases where a user is being blocked or rate limited by Cloudflare. The absence of logs in Kibana or RateLimit- headers is usually an indication to investigate at the Cloudflare level. You can typically request a screenshot of the Cloudflare "Access Denied" page or have the customer perform a curl with the -i flag to retrieve the relevant headers:

text

```
curl -i --header "PRIVATE-TOKEN: " https://gitlab.com
```

HTTP/2 403

date: Thu, 18 Nov 2021 13:48:09 GMT

content-type: text/plain; charset=UTF-8

content-length: 16

x-frame-options: SAMEORIGIN

referrer-policy: same-origin

cache-control: private, max-age=0, no-store, no-cache, must-revalidate, post-check=0, pre-check=0

expires: Thu, 01 Jan 1970 00:00:01 GMT

expect-ct: max-age=604800, report-uri="https://report-uri.cloudflare.com/cdn-cgi/beacon/expect-ct"

report-to:

{"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v3?s=AbCdeFgXXXXX"}],"group":"cf-nel","maxage":604800}

nel: {"successfraction":0.01,"reportto":"cf-nel","maxage":604800}

strict-transport-security: max-age=31536000

x-content-type-options: nosniff

server: cloudflare

cf-ray: 6b01a12345abcde-KBP

error code: 1020

Note the HTTP 403 response and error code 1020. You should also take note of the cf-ray ID to use in the Cloudflare Dashboard.

Once you obtain this information, you should follow our guide for troubleshooting Cloudflare. Some blocks may happen as a result of a mitigation effort, so you may want to verify that a contact request is not open on the internal board.

Note that IP addresses may be blocked if they are identified as being from a current US embargoed country as per our Terms of Use. Blocks are done automatically through Cloudflare's GeoLocation block methods and cannot be changed. You can enter an IP address to determine how it is classified and verify against the list of countries. A user can consider requesting a data correction of their IP address but it is not guaranteed and GitLab has no control over this process.

## Bypass Policy

If a customer has concerns about being rate limited, work with them as much as possible to lower their traffic from a single IP address.

Please refer to our Bypass Policy for more information.

---

## SECTION: support/workflows/jira-server.md

## What is Jira Software

Jira Software is part of a family of products designed to help teams of all types manage work. Originally, Jira was designed as a bug and issue tracker. But today, Jira has evolved into a powerful work management tool for all kinds of use cases, from requirements and test case management to agile software development.

For more information about various uses of Jira Please check out the Get started with Jira Software

## How to Set Up Jira Server

1. Create a new host using GitLab Sandbox Cloud for your Jira Server installation. This should be separate from your GitLab instance. Ensure port 443 is available on this new host, as we will set up Jira to use HTTPS for GitLab integration.

## Prerequisite software

- You need to install Java as Jira uses java JVM.
- You need to install certbot, python3-certbot-apache for you to generate certificates for HTTPS

## Installing Jira and prerequisites

1. Install Java because Jira uses Java JVM

```
bash
apt install openjdk-11-jdk openjdk-11-jre
```

1. Create a folder to store Jira software for installation.



```
bash
mkdir jira && cd jira
```

1. Choose the version of Jira that you want to download from Atlassian Jira Website.

We will be using the Jira 8.13 version so that we can test all supported GitLab integrations. However, this will not work for OAuth2.0 integration so we will later upgrade Jira to test the connection.

1. Select the Jira version that you want to download.

1. Accept and right-click submit button to get the download URL. (Choose copy link address during the file download)

1. Use wget <URL> command to download the Jira installer. Example:

```
bash
wget https://www.atlassian.com/software/jira/downloads/binary/atlassian-jira-
software-8.13.7-x64.bin
```

1. Change the script to be executable using

```
bash
chmod a+x atlassian-jira-software-8.13.7-x64.bin
```

1. Run the installer and enter Y if it requests you to install OpenJDK.

```
bash
sudo ./atlassian-jira-software-8.13.7-x64.bin
```

1. Follow the installation instructions and enter default for all questions asked in the installation.

1. Run Jira setup. You will activate a 30 day trial period. Select set up Jira manually.

1. You now have Jira setup and accessible in your localhost:8080.

1. You will have to set up Jira before using it. After you access from the browser, Select I'll set it up myself.

1. In the next step, select Built-in (for evaluation or demonstration).

1. While waiting for the database to be created, login to your atlassian account and open <https://my.atlassian.com/product> . Generate a license trial license for your server.

1. On the next page, select Jira Software for Product and Jira Software (Data Center) for License type. Enter any details for the remaining fields. For Server ID, you will get it when the server finishes setting up the database.

1. After the database creation is complete, the next section will be setting application properties. Leave it as default and click Next.

![Jira Application Properties.png]

1. In the next scene, specify your license Key. Copy the Server ID and paste it to the New trial License Generation screen and click on Generate License.

1. Copy and paste the license key generated and update license in your Jira setup.

1. Create a Jira Admin user and continue with the next steps.

1. Continue with the setup and create a test project. "GITLAB".

1. Create a test issue that we will be using to test with.

Adding a lets-encrypt certificate to enable HTTPS connection

NOTE:

HTTPS connection is required for DVCS Connector

1. Install certbot to enable you to generate the certificate

```
bash
sudo apt-get update && apt-get upgrade
sudo apt-get install certbot
```

Optionally, you can install the Certbot Apache plugin:

```
bash
sudo apt-get install python3-certbot-apache
```

1. Generate a certificate for your domain. Input your e-mail when prompted.

```
bash
sudo certbot certonly --standalone -d www.example.com
```

1. If everything goes fine. A new SSL will be issued at the below location. Navigate to the below directory and view files.

```
bash
cd /etc/letsencrypt/live/example.com
ls
```

You should be able to see the following list:

- cert.pem
- chain.pem
- fullchain.pem
- privkey.pem

**NOTE:**

In case you get a Problem binding to port 80: Could not bind to IPv4 or IPv6. error, check the PID of the application using port 80 then stop the application running on that port and retry generating the certificates.

```
bash
Check with lsof
lsof -i :80
```

You can also use  
netstat -antlup | grep 80

If it's Apache, you can run the following:  
systemctl stop apache2  
systemctl disable apache2

Convert keypair and certificate to Java Keystore

**NOTE:**

I will be using dwainaina-gitlab-jira-test-runner.sr.gitlab.support as my domain for the rest

of the examples.

1. Create a PKCS12 that contains both your full chain and the private key. You need to have OpenSSL installed for that.

```
text
openssl pkcs12 -export -out /tmp/dwainaina-gitlab-jira-test-runner.sr.gitlab.support.p12 \
  -in /etc/letsencrypt/live/dwainaina-gitlab-jira-test-
runner.sr.gitlab.support/fullchain.pem \
  -inkey /etc/letsencrypt/live/dwainaina-gitlab-jira-test-
runner.sr.gitlab.support/privkey.pem \
  -name tomcat
```

Replace the values accordingly. For name, we put it tomcat you may change the alias to anything you like but please remember this name.

1. Now convert your .p12 file to a .jks file with the command below. Replace <your-domain-name> with your domain name, and <password> with the password you used to generate your .p12 file:

```
bash
keytool -importkeystore \
  -deststorepass <password> -destkeypass <password> -destkeystore /tmp/<your-domain-name>.jks \
  -srckeystore /tmp/<your-domain-name>.p12 -srcstoretype PKCS12 -srcstorepass <password> \
  -alias tomcat
```

1. Copy the generated JKS file to /opt/atlassian/jira/conf

```
bash
cp /tmp/dwainaina-gitlab-jira-test-runner.sr.gitlab.support.jks /opt/atlassian/jira/conf/
```

1. Change directory to /opt/atlassian/jira/conf/ and edit the server.xml. Before editing your server.xml file, consider backing it up first, just in case you need to quickly roll back.

```
bash
cd /opt/atlassian/jira/conf
cp server.xml serverbackup.xml
vim server.xml
```

1. Comment out to disable HTTP.

```
xml
<!--
  <Connector port="8080" relaxedPathChars=""
relaxedQueryChars=""{}^&x5c;&x60;&quot;&lt;&gt;"
    maxThreads="150" minSpareThreads="25" connectionTimeout="20000"
```

```

enableLookups="false"
    maxHttpHeaderSize="8192" protocol="HTTP/1.1" useBodyEncodingForURI="true"
redirectPort="8443"
    acceptCount="100" disableUploadTimeout="true" bindOnInit="false"/>
-->

```

1. Add the following just before the closure of the </Service> block in server.xml

```

xml
<Connector port="443" protocol="org.apache.coyote.http11.Http11NioProtocol"
    maxHttpHeaderSize="8192" SSLEnabled="true"
    maxThreads="150" minSpareThreads="25"
    enableLookups="false" disableUploadTimeout="true"
    acceptCount="100" scheme="https" secure="true"
    sslEnabledProtocols="TLSv1.2,TLSv1.3"
    clientAuth="false" useBodyEncodingForURI="true"
    relaxedPathChars="[]" relaxedQueryChars="[]|{}^&x5c;&x60;&quot;&lt;&gt;"
    keyAlias="tomcat" keystoreFile="conf/dwainaina-gitlab-jira-test-
runner.sr.gitlab.support.jks" keystorePass="JiraPASSWORD" keystoreType="JKS" />

```

Replace the values for your keyAlias, keystoreFile and keystrokePass accordingly.

1. Edit the web.xml in conf folder and add the following so that all requests will be handled using the HTTPS. Again, before editing your web.xml file, consider backing it up first, just in case you need to quickly roll back.

```

bash
cd /opt/atlassian/jira/conf
cp web.xml webbackup.xml
vim web.xml

```

Add the following lines right after the <web-app ... version="3.1"> tag:

```

xml
<security-constraint>
  <web-resource-collection>
    <web-resource-name>all-except-attachments</web-resource-name>
    <url-pattern>.jsp</url-pattern>
    <url-pattern>.jspx</url-pattern>
    <url-pattern>/browse</url-pattern>
    <url-pattern>/issues</url-pattern>
  </web-resource-collection>
  <user-data-constraint>
    <transport-guarantee>CONFIDENTIAL</transport-guarantee>
  </user-data-constraint>
</security-constraint>

```

1. Optional: Specify Jira home directory if it is not at default location:

```
bash
vim <installation-directory>/atlassian-jira/WEB-INF/classes/jira-application.properties
jira.home = /PATH/TO/JIRA-HOME
```

1. Restart tomcat and check if the site is available in HTTPS. If you can't find a service with the name tomcat, restart the jira service:

```
bash
systemctl status jira
systemctl restart jira
systemctl status jira
```

NOTE:

HTTPS port might not be accessible after you restart tomcat. Run the following command to enable port and apply settings:

```
bash
echo 'net.ipv4.ipunprivilegedportstart=0' > /etc/sysctl.d/50-unprivileged-ports.conf
sysctl --system
```

1. The site should now be available as a HTTPS address. Use the credentials you used to configure to now log in.

NOTE:

If you still can't access the site using HTTPS, check the permissions of your `/opt/atlassian/jira/conf/<your-domain-name>.jks` file. Every user should be able to at least read the file. Replace `<your-domain-name>` with your domain name.

```
bash
ll /opt/atlassian/jira/conf/<your-domain-name>.jks
```

If you need to change permissions, set it to at least 444 and restart the jira service

```
chmod 444 /opt/atlassian/jira/conf/<your-domain-name>.jks
```

Restart the jira service for changes to take effect

```
systemctl status jira
systemctl restart jira
systemctl status jira
```

## Common Troubleshooting Steps for Jira tickets

1. Please check out the common known errors listed in our documentation: [Jira - Troubleshooting and Troubleshooting your DVCS connection](#)

1. Clarify with customers with integration they are referring to - we have GitLab Jira Integration and Jira Development Panel integration.

1. It is also helpful to get their Jira version - particularly: Cloud or Server? Is Server which version? (Note: 8.14 later links differently)

1. Check their configuration, walk through the steps.

NOTE:

Some customers confused the configuration of the two integrations, for example using the jira user created in GitLab Jira Integration as the Term/Account for setting up DVCS account for their Jira Development Panel. This will load the repo of the jira user's personal namespace which usually has 0 repos.

1. If all the configurations are correct, we might need to get the logs depending on where the error is occurring, they might need to check the log files on Jira, they can tail JIRAHOME/log/atlassian-jira.log while reproducing the error. If it's Jira DVCS connection has wrong oauth scope: Issue when integrating with Jira DVCS, check out the workaround.

1. We also have a list of useful Rails cheat code, this is handy when we need to check the values of the Jira configurations on GitLab.

---

## SECTION: support/workflows/kibana.md

## Overview

This document provides information on what Kibana is, how to search it, interpret its results, and contains tips and tricks on getting specific information from it.

## Using Kibana

Kibana is an open source data visualization plugin for Elasticsearch. It provides visualization capabilities on top of the content indexed on an Elasticsearch cluster. Support Engineering uses Kibana to both search for error events on GitLab.com and to detect when specific changes were made to various aspects of it by a user.

>Note: Kibana defaults to the UTC time zone. It does not retain logs older than 7 days. If you're working on a ticket where access to older logs would have been helpful, please flag it via the Support::SaaS::Log retention period reached macro (This is an internal macro for tracking purposes only).

## Parameters

Knowing where to search in Kibana is paramount to getting the proper results. The area of the application (GitLab.com) that you're searching is known as the Index in Kibana. The default index used for searching is pubsub-rails-inf-gprd- but you can change this by clicking the (change) button:

Indexes closely correlate for the most part with our log structure in general. Some other frequently used indexes are:

- pubsub-gitaly-inf-gprd-
- pubsub-pages-inf-gprd-
- pubsub-runner-inf-gprd-

For example, if you're trying to track down failed logins you would search the index pubsub-rails-inf-gprd-. To search for 500 errors involving a controller you'd search in pubsub-rails-inf-gprd-, the default index.

Along with the index, knowing when a specific error or event occurred that you're trying to track down is important and it's important to keep in mind that Kibana logs on GitLab.com persist for seven days. Kibana allows you to choose relative and absolute time ranges for search results and this can be changed by manipulating the date range:

## Fields and Filters

Note: As of March 2022, using Filters is the recommended way for searching Kibana. Using the general search box is discouraged as it may generate several errors in the form of X of Y Shards Failed. See Infra Issue.

Each log entry is comprised of a number of Fields in which the specific information about the entry is displayed. Knowing which fields to apply to your search results and how to filter for them is just as important as knowing where and when to search. The most important fields are:

- json.method
- json.action
- json.controller - for more on this see the controller definitions in the GitLab sourcecode
- json.status
- json.path

All available fields are displayed along the left-hand side menu and you can add them to your search results by hovering over each and clicking the add button.

If you don't filter for specific fields it can be difficult to find specific log entries if a large number of them are returned on your search query.

For example, we're trying to locate any log events generated by the GitLab.com user tristan that returned a 404 status code within the last 15 minutes. We can start by searching the pubsub-rails-inf-gprd- index for json.username : tristan within that time range and we'd get results similar to the following once we click add next to the json.status field along the left-hand side bar:

The majority of results as entries that returned 200, which aren't in the scope of what we're looking for. To search for results where 404 was returned we can click + Add Filter underneath the search bar and place a positive filter on json.status is 404, which will give us these



results, exactly what we're looking for.

### Identify cause of IP Blocks

There are some useful tips here about searching kibana for errors related to IP blocks.

### Log Identification

Not sure what to look for? Consider using a Self-Managed instance to replicate the bug/action you're investigating. This will allow you to confirm whether or not an issue is specific to GitLab.com, while also providing easily accessible logs to reference while searching through Kibana.

Support Engineers looking to configure a Self-Managed instance should review our Sandbox Cloud page for a list of available (company provided) hosting options.

### Sharing logs

To share the current state of your log search, follow Elastic's log sharing documentation. Copying the URL directly will fail to load your search when other users attempt to use it. Since it is encoded, your search parameters will not be included in the URL.

### Dashboards

There are a number of available dashboards that are designed to create visualizations that can help in identifying patterns. It can also be useful for narrowing in on specific areas of the product or a identifying a specific issue or bug. While any GitLab team member can create a dashboard that can be used by anyone, there are a few support specific dashboards:

- Correlation Dashboard
- Rate Limit & Requests Dashboard
- Integrations Dashboard
- Errors & Exceptions Dashboard

You can access dashboards in Kibana under Analytics > Dashboard and filtering by the Support tag or by searching for the name of the dashboard.

If you are interested in creating your own dashboard, you might want to consider reading Elastic's guide on creating dashboards. If you've created something that might have a wider use, consider adding the Support tag to your dashboard so it is easily found by other support team members.

To use dashboards, be sure you are in View mode. In view mode, you can input values and it will update the visuals accordingly but not save it to the dashboard. Use Share > Get Links if you need to share the snapshot of what you captured.

Before editing an existing dashboard, consider cloning it to make your own copy and label it accordingly.

## Correlation Dashboard

If you found a Correlation ID that is relevant for your troubleshooting, you can utilize the correlation dashboard to quickly view all related components across indices without having to search each individual index. Once you view the dashboard, simply click on the `json.correlationid` filter and enter in your found correlation ID. It will then search across w